

Final Project Report

Felipe Oscanoa

ECE 528L

Project Overview:

The objective of this project is to design and implement a remotely controlled (RC) car using an ESP32 microcontroller. The system will integrate a wireless control interface and an onboard camera to provide real-time video feedback to the user. The final system will allow a user to remotely drive the vehicle while simultaneously viewing live video from the car's perspective.

System Architecture:

The architecture used for this project will consist of making the RC Car around a chassis that will be fully 3d printed, so it will be able to get literally unlimited spare parts for this project. The main brain for this Car will be the ESP32, since this will give many possibilities to make this project work, along with an extended room for possible improvements in the future. Along with the ESP32, the other components are gonna be referenced for other RC Car builds, like the ESC, motor, and Servo. But for this project, I am going for a more budget side and going for the cheapest option that will be all brushed motor, brushed ESC, and the regular servo. Additionally, to power all the system, a Zee Lypo Battery 2s of 5200Mhamh to give power to the whole system and for it to be fast.

Lastly, an ESP32 CAM is implemented to have a way of seeing what the car is seeing, and have the feeling that I would be driving the car in first person.

Interfaces and Peripherals Used:

For this part, I used the Arduino IDE since it is mostly easier to work in here because of the facility of adding libraries and adding the main files for the ESP32. For the board initializations, I used the ESPRESSIF ESP32 board files that are available on the Arduino IDE. Following by taking reference from a YouTube video, I added the library of ESP32 PS5 to establish a connection between the PS5 controller and the ESP32 by using the MAC number of the controller, so that anytime the ESP32 is started, it will directly connect to this controller. The ESP32 was used to get the servo motor, along with the ESC, started and ready to be controlled by the PS5 controller, since in this. library, there are initializations for both of these devices, so it was just a question of using the right thresholds and values for the forward and reverse. In the part of the ESC, there was a conflict that is because the ESC has a bigger capacity for giving power to the forward movement and less power to the reverse, which, instead of being solved it was just worked around by adjusting the values.

For the part of the servo motor, it was just a matter of tuning it, since if it was given the full range from 0 to 180, it would hit the walls, so it was reduced to the right values so it would not hit them. Lastly, the camera was started by using the CameraWebServer, which is a library that has the initialization for the ESP32 camera, so it will either connect to a WIFI or make its own, so the computer or other device will connect to it, and then using the ip address given by the ESP32 CAM we can access it by entering into the ESP32 OV3660 website to see the camera.

Verification and Testing:

In this part, for the verification of the code working, was just first getting everything, the simplest parts first, which was the correct read of inputs from the PS5 controller to the ESP32 by getting the mapping of values printed whenever we see a change by using the controller. This was tested for the steering and the motor activation of moving forward using the L2 button and the R2 for reverse. After this step, was adding the servo in place since it could just be powered by the same ESP32. We can see that the mapping of these values affects the movement in the servo. Following by using the battery and the ESC, we will power up the DC motor, and using a buck converter, we lower the output of the ESC from 6V to 5V, this to power up the ESP32 too. After this, I did the same procedure of mapping to see the forward and reverse activation of the DC motor.

Finally, after having these parts done, I just placed everything in place and have both parts in the 3d printed chassis and started moving around. Additionally, since the car was moving correctly, it was just a question of adding the ESP32 CAM on the front and soldering a micro USB cable using the output from the buck converter. I was able to power up the ESP32 CAM and see the live video.

Project Demonstration:

For this part, I made a YouTube video; the link is <https://www.youtube.com/watch?v=UeRciERhonk>.

Additionally, there are 2 more videos added to this repository that have some other footage of the car running around.

Conclusion:

For this part, I was able to achieve the correct working of the RC car by using the ESP32 as a communication device and being able to make it connect to the ESC and servo for drivability. Additionally, with the camera, I was able to see live footage of the car when it was driving, although it was very laggy and not stable. This could be caused because the camera is making its own wifi and we connect to the computer to it, which causes instability the further or faster it goes.

This could be solved by using a common wifi in the place so both devices will be connected to it and to have a more precise driving. In part for the chassis, since it is printed with

various types of material like ASA-CF, PETG, PLA-CF, and TPU. I was able to accomplish a more stable driving experience with it, not having melting parts or broken parts that often, but it will definitely be nice and more efficient to have CNC parts or real carbon fiber parts. Finally, thanks to this project, I was able to get a better understanding of microprocessors like the ESP32 and its capabilities since I have barely covered the many other functions and ways of using the ESP32, which I will continue to approach to get a better knowledge of it, and get more upgrades for this car as well.